GENAI PROJECT PORTFOLIO

Six Python Projects — RAG, Embeddings, Chatbots & E-commerce AI

1. Enterprise Knowledge Assistant (RAG)
2. Semantic Product Search & Recommendation Engine (Embeddings)
3. AI Shopping Assistant Chatbot for E-commerce
4. Support Ticket Triage & Auto-Response Chatbot
5. AI Meeting Summarizer & Action-Item Tracker
6. AI Resume Screener & Job-Matching Assistant

All projects are implemented in Python.

PROJECT 1

Enterprise Knowledge Assistant

Stack: Python | Focus: Retrieval-Augmented Generation (RAG)

Problem Statement

Employees waste significant time searching through scattered HR policies, IT procedures, and internal handbooks (PDFs, Word docs, wikis) to answer simple questions such as leave policy, reimbursement rules, or password-reset steps. Build a Python-based assistant that ingests these internal documents, retrieves the most relevant passages for a user's natural-language question, and generates a grounded, cited answer — instead of a generic or hallucinated response.

Working Flow

- **Ingest:** Load internal documents (PDF/DOCX/TXT) and split them into overlapping text chunks.
- **Embed:** Convert each chunk into a vector embedding using an embedding model.
- **Store:** Persist embeddings + metadata (source file, section) in a vector database (e.g. FAISS/Chroma).
- **Retrieve:** On a user query, embed the question and fetch the top-k most similar chunks.
- **Generate:** Pass the retrieved chunks + question to an LLM to produce a grounded answer.
- **Respond:** Return the answer along with the source document/section as a citation.

Features Needed

- Document ingestion pipeline supporting PDF, DOCX, and plain text.
- Chunking strategy with configurable chunk size and overlap.
- Vector store integration (FAISS, Chroma, or similar) for similarity search.
- Prompt template that forces the LLM to answer only from retrieved context.
- Source citation displayed alongside every answer.
- Simple chat-style interface (CLI, Streamlit, or Flask) for asking questions.

Evaluation

Criterion	Weight	What SME checks
Retrieval Quality	25%	Correct, relevant chunks are retrieved for varied queries
Answer Grounding	25%	Answers are based on retrieved content, not hallucinated
Citation Accuracy	20%	Cited source actually supports the given answer
Chunking & Embedding Setup	15%	Sensible chunk size, overlap, and embedding choice
Usability & Documentation	15%	Working interface, clear setup instructions

PROJECT 2

Semantic Product Search & Recommendation Engine

Stack: Python | Focus: Embeddings

Problem Statement

Traditional keyword search on an e-commerce catalog fails when a customer's query doesn't use the exact words in a product listing (e.g., searching "cozy winter jacket for kids" misses a product titled "Children's Insulated Parka"). Build a Python system that embeds the product catalog (titles, descriptions, attributes) into vector space so search understands meaning, and additionally recommends similar products using vector similarity.

Working Flow

- **Prepare Catalog:** Clean and normalize product titles, descriptions, and attributes.
- **Embed Products:** Generate a vector embedding for each product's combined text fields.
- **Index:** Store embeddings in a vector index for fast nearest-neighbor lookup.
- **Query:** Embed the user's search query and retrieve top-N nearest products by similarity.
- **Recommend:** For any given product, fetch its nearest neighbors as "similar products."
- **Rank & Return:** Combine similarity score with basic business rules (stock, rating) before returning results.

Features Needed

- Catalog preprocessing script (cleaning, normalization, deduplication).
- Embedding generation pipeline for all product records.
- Vector similarity search (FAISS/Annoy/Chroma) with top-N retrieval.
- Search API/endpoint accepting free-text natural-language queries.
- "Similar products" recommendation feature for any given product ID.
- Basic re-ranking logic combining similarity with rating/stock availability.

Evaluation

Criterion	Weight	What SME checks
Search Relevance	30%	Natural-language queries return semantically correct products
Recommendation Quality	25%	"Similar products" are genuinely related
Embedding & Indexing Setup	20%	Efficient, correctly configured vector index
Re-ranking Logic	15%	Sensible use of business signals alongside similarity
Code Quality & Documentation	10%	Clean, reproducible, well-documented pipeline

PROJECT 3

AI Shopping Assistant Chatbot

Stack: Python | Focus: E-commerce AI

Problem Statement

Shoppers on an e-commerce site often need guidance — comparing products, understanding specs, or getting recommendations — but static filters and search bars don't hold a conversation. Build a Python-based conversational shopping assistant that helps users find products, compares options, answers spec questions, and gives personalized suggestions, grounded strictly in real product data so it never invents prices or specifications.

Working Flow

- **Understand Intent:** Parse the user's message to detect intent (search, compare, ask spec, recommend).
- **Fetch Data:** Query the product database/catalog for relevant items matching the intent.
- **Ground Response:** Pass the fetched product data + user message to an LLM to generate a natural reply.
- **Personalize:** Factor in the user's browsing/purchase history (if available) for recommendations.
- **Clarify When Needed:** If the request is ambiguous (e.g., budget/size not given), ask a follow-up question.
- **Respond & Continue:** Return the answer and keep conversation context for follow-up turns.

Features Needed

- Intent classification for search / compare / spec-question / recommendation requests.
- Integration with product database to ground every answer in real data.
- Multi-turn conversation memory (context carried across messages).
- Comparison logic that generates side-by-side feature summaries.
- Personalization using simple browsing/purchase history signals.
- Clarifying-question flow for incomplete or ambiguous requests.

Evaluation

Criterion	Weight	What SME checks
Grounding & Accuracy	30%	No hallucinated prices/specs; all answers match catalog data
Conversational Flow	20%	Natural multi-turn dialogue, correct context retention
Recommendation Relevance	20%	Personalized suggestions are genuinely useful
Handling Ambiguity	15%	Clarifying questions asked when input is incomplete
Code Quality & Documentation	15%	Clean structure, working demo, clear setup

PROJECT 4

Support Ticket Triage & Auto-Response Bot

Stack: Python | Focus: Applied NLP / Chatbot

Problem Statement

A support team receives a high volume of tickets of mixed urgency and topic, and manually sorting and responding to each one is slow and inconsistent. Build a Python system that automatically classifies incoming tickets by intent and urgency, drafts an auto-response for common, well-understood issues, and escalates complex or high-priority tickets to a human agent with a pre-filled summary.

Working Flow

- **Ingest Ticket:** Receive incoming ticket text (subject + body) from a form, email, or API.
- **Classify:** Use an LLM or trained classifier to tag the ticket's category and urgency level.
- **Decide Path:** If the category is common/low-risk, proceed to auto-response; otherwise escalate.
- **Auto-Respond:** Generate a draft reply grounded in a knowledge base of past resolutions/FAQs.
- **Escalate:** For complex/urgent tickets, generate a concise summary and route to a human agent.
- **Log & Track:** Store the classification, action taken, and outcome for reporting.

Features Needed

- Ticket classification model/pipeline for category and urgency.
- Knowledge base lookup (FAQ/past resolutions) to ground auto-responses.
- Auto-draft generation for common issue types.
- Escalation logic with automatic summary generation for human agents.
- Dashboard or log view showing triage outcomes across tickets.
- Confidence threshold to avoid auto-responding when classification is uncertain.

Evaluation

Criterion	Weight	What SME checks
Classification Accuracy	25%	Correct category/urgency assigned across varied tickets
Auto-Response Quality	25%	Drafted replies are accurate, relevant, and well-grounded
Escalation Logic	20%	Complex/urgent tickets correctly routed with useful summaries
Confidence Handling	15%	Uncertain cases are not auto-answered incorrectly
Code Quality & Documentation	15%	Clean pipeline, reproducible, clearly documented

PROJECT 5

AI Meeting Summarizer & Action-Item Tracker

Stack: Python | Focus: GenAI Productivity Tool

Problem Statement

Teams routinely lose track of what was discussed and agreed upon in meetings because notes are inconsistent or never taken. Build a Python tool that takes a raw meeting transcript, generates a clear summary, extracts action items with owners and deadlines, and allows users to ask follow-up questions about past meetings using a lightweight retrieval layer over stored transcripts.

Working Flow

- **Input Transcript:** Accept a meeting transcript (text file or speech-to-text output).
- **Summarize:** Use an LLM to generate a concise summary of key discussion points.
- **Extract Actions:** Prompt the LLM to pull out action items, each with an owner and due date if mentioned.
- **Store:** Save the transcript, summary, and action items (embedded for later retrieval).
- **Query Past Meetings:** Embed follow-up questions and retrieve relevant past-meeting content to answer them.
- **Track:** Maintain a simple action-item list/status view across meetings.

Features Needed

- Transcript ingestion (plain text, with optional speech-to-text integration).
- Summarization pipeline producing a structured, concise meeting summary.
- Action-item extraction returning structured data (task, owner, deadline).
- Vector store of past meeting transcripts/summaries for follow-up Q&A (light RAG).
- Action-item tracker view showing status (open/done) across meetings.
- Export option for summaries and action items (e.g., to text/markdown).

Evaluation

Criterion	Weight	What SME checks
Summary Quality	25%	Accurate, concise summary capturing key discussion points
Action-Item Extraction	25%	Correctly identifies tasks, owners, and deadlines
Follow-up Q&A (RAG)	20%	Relevant, grounded answers about past meetings
Tracking & Usability	15%	Action items are trackable and easy to review
Code Quality & Documentation	15%	Clean structure, reproducible, clearly documented

PROJECT 6

AI Resume Screener & Job-Matching Assistant

Stack: Python | Focus: Embeddings + RAG

Problem Statement

Recruiters manually screening large volumes of resumes against job descriptions is slow and inconsistent. Build a Python system that embeds resumes and job descriptions into vector space, ranks candidates by similarity to a given job, and uses an LLM to generate a plain-language explanation of why a candidate is (or isn't) a good fit — highlighting matched skills and gaps.

Working Flow

- **Parse Documents:** Extract text from resumes (PDF/DOCX) and job descriptions.
- **Embed:** Generate vector embeddings for each resume and each job description.
- **Match:** Compute similarity scores between a job description and all candidate resumes.
- **Rank:** Shortlist the top-N candidates by similarity score for a given role.
- **Explain:** For each shortlisted candidate, prompt an LLM (grounded in resume + JD text) to explain fit.
- **Report:** Output a ranked shortlist with fit explanations for the recruiter to review.

Features Needed

- Resume and job description parsing (PDF/DOCX to clean text).
- Embedding generation for resumes and job descriptions.
- Similarity-based ranking/shortlisting logic.
- LLM-generated fit explanation grounded in the actual resume and JD text.
- Skill-gap highlighting (matched vs. missing skills) in the explanation.
- Exportable shortlist report (e.g., CSV or markdown) for recruiters.

Evaluation

Criterion	Weight	What SME checks
Matching Accuracy	30%	Similarity ranking reasonably reflects true candidate fit
Explanation Quality	25%	Fit explanations are accurate, grounded, and clear
Skill-Gap Identification	20%	Matched and missing skills are correctly identified
Parsing Robustness	15%	Handles varied resume/JD formats without breaking
Code Quality & Documentation	10%	Clean pipeline, reproducible, clearly documented